

	EMPRESA DE DESARROLLO E INNOVACIÓN	CÓDIGO: MI-OPE-ESP-001
	PROCESO: TECNOLOGÍA, INFORMÁTICA Y COMUNICACIONES	VERSIÓN: 1.0
	OPERACIONES	ELABORACIÓN: 7/10/2025
	Informe Técnico – Módulo de Autenticación	ÚLTIMA REVISIÓN: 7/10/2025

1. Introducción

El presente documento describe la implementación del **Módulo de Autenticación** desarrollado para el sistema **Moverly**, plataforma digital para la gestión de mudanzas y fletes. Este componente garantiza la correcta gestión de identidad, acceso y trazabilidad de los usuarios, permitiendo diferenciar los roles de cliente, conductor y administrador dentro del ecosistema del proyecto.

El módulo asegura la integridad del proceso de autenticación mediante el uso de **JSON Web Tokens (JWT)**, dependencias seguras de **FastAPI** y encriptación de contraseñas con **bcrypt**, cumpliendo con buenas prácticas de desarrollo seguro.

2. Objetivo

Diseñar e implementar un sistema de autenticación robusto que:

- Permita el registro y acceso de clientes, conductores y administradores de forma segura.
- Emita y valide tokens JWT con expiración controlada.
- Restrinja el acceso a rutas protegidas según el rol del usuario.
- Almacene las credenciales de manera segura mediante hashing.

3. Alcance

El módulo incluye:

- Registro de usuarios con validación de unicidad de correo electrónico.
- Inicio de sesión con generación de **tokens JWT**.
- Validación y autorización basada en roles.
- Dependencias de autenticación integradas a los endpoints de FastAPI.
- Validaciones mediante **Pydantic** y manejo de errores estándar (HTTPException).

4. Definiciones

Término	Definición
---------	------------

JWT	Token de autenticación sin estado usado para validar identidad y permisos de usuario.
bcrypt	Algoritmo de hashing utilizado para proteger contraseñas antes de almacenarlas.
Rol	Conjunto de permisos que determinan el nivel de acceso (cliente, conductor o admin).
Endpoint	Ruta HTTP del API que ejecuta una operación definida en el backend.
Token Bearer	Cabecera HTTP que transporta el JWT en cada solicitud autenticada.

5. Responsabilidades

Responsable	Descripción de la responsabilidad
Kevin Sánchez	Coordinación general del módulo, planificación de tareas y pruebas funcionales.
Patricio Salazar	Implementación del backend, autenticación JWT, middlewares y seguridad.
Angie Caiza	Validación visual en Swagger, apoyo en pruebas y documentación complementaria.
Thomas Vaca	Redacción del informe técnico, recopilación de evidencias y control de versiones.

6. Normativa Legal Aplicable

- **Ley Orgánica de Protección de Datos Personales (Ecuador, 2021):** Establece normas para la protección de información sensible de usuarios.
- **Código Ingenios (2016):** Regula la propiedad intelectual y uso de software libre en Ecuador.
- **ISO/IEC 27001:** Seguridad de la información y gestión de accesos.
- **ISO/IEC 25010:** Estándar de calidad de software, con énfasis en fiabilidad y seguridad.
- **OWASP Top 10:** Buenas prácticas para mitigar vulnerabilidades comunes (inyección, XSS, autenticación rota, etc.).

7. Método / Políticas / Contenido

Metodología

El módulo se desarrolló dentro del **Sprint 2** del proyecto, aplicando metodología **Scrum** con sprints semanales, planificación, revisiones y retrospectivas.

Políticas de Desarrollo

- Uso de **GitHub** con estrategia de ramas **main** y **develop**.
- Principios **SOLID** y arquitectura limpia para separación de capas.
- Estilo de código PEP 8 y commits convencionales (**feat:**, **fix:**, **docs:**).

Políticas de Seguridad

- Hashing de contraseñas con bcrypt (12 rounds).
- JWT con expiración de 24h y firma HS256.
- Variables de entorno (.env) para credenciales sensibles.
- Validación de datos con Pydantic.

Cronograma de Actividades

Actividad	Fecha	Responsable	Entregable	
Diseño del módulo de autenticación	29/09/2025	Kevin Sánchez	Plan de endpoints y roles definidos.	
Implementación de registro de usuario	30/09/2025	Patricio Salazar	Endpoint <code>/auth/register</code> operativo.	
Implementación de login y generación JWT	01/10/2025	Patricio Salazar	Endpoint <code>/auth/login</code> funcional.	
Desarrollo de validación de roles (RBAC)	02/10/2025	Kevin Sánchez	Middleware de roles configurado.	
Pruebas en Swagger UI	03/10/2025	Angie Caiza	Evidencias de registro, login y token válidas.	
Documentación y redacción del informe	04/10/2025	Thomas Vaca	Informe técnico consolidado.	

Cuadro 3: Cronograma y actividades del módulo de autenticación.

8. Arquitectura y Componentes

El módulo se encuentra implementado bajo la arquitectura SOA ligera definida para el proyecto. Los principales componentes y archivos son:

Componente / Archivo	Descripción
<code>auth_router.py</code>	Define las rutas de registro, login y validación de tokens.
<code>auth_service.py</code>	Contiene la lógica de negocio para creación, verificación y autenticación de usuarios.
<code>security.py</code>	Maneja la creación y validación de JWT, y el hashing con bcrypt .
<code>schemas/auth.py</code>	Define los modelos Pydantic para solicitudes y respuestas (login, registro, token).
<code>models/usuarios.py</code>	Estructura la tabla Usuarios y sus relaciones con la base de datos.
<code>middlewares/role_guard.py</code>	Controla la autorización por rol usando dependencias de FastAPI.

9. Flujo de Autenticación

Registro de Usuario

1. El usuario envía una solicitud POST a `/auth/register` con sus datos (nombre, correo, contraseña, rol).
2. Se valida que el correo no exista previamente.
3. La contraseña se transforma a hash mediante **bcrypt**.
4. Se almacena el usuario en la base de datos.
5. El sistema responde con un token JWT válido y el rol asignado.

Inicio de Sesión

1. El usuario envía una solicitud POST a `/auth/login`.
2. Se valida el correo y la contraseña mediante `verify_password`.
3. Si la autenticación es exitosa, se genera un token con expiración controlada.
4. El sistema retorna el token junto con los datos del usuario autenticado.

Pseudodiagrama de Flujo

Registro:

```

validar correo único
hash contraseña
insertar en BD
devolver JWT

```

Login:

```

validar credenciales
generar JWT
devolver JWT

```

10. Endpoints Documentados (Swagger)

Endpoint	Descripción	Restricciones / Rol
POST /auth/register	Crea un nuevo usuario y retorna un JWT	Público (cliente o conductor)
POST /auth/login	Inicia sesión y genera un JWT	Público
POST /auth/token	Soporte OAuth2 (Swagger “Authorize”)	Público
GET /auth/me	Retorna datos del usuario autenticado	Token requerido
GET /usuarios	Lista todos los usuarios registrados	Solo admin

11. Manejo de Roles y Autorización

- Los roles válidos son: **cliente**, **conductor** y **admin**.
- La verificación se realiza mediante dependencias de FastAPI (`Depends(validate_admin)`).
- Los endpoints administrativos no pueden ser accedidos por usuarios sin el rol **admin**.
- Los intentos de creación de administradores desde el registro público son rechazados.

12. Validaciones y Seguridad

- Contraseñas almacenadas con **bcrypt**.
- Tokens JWT con expiración de 24 horas.
- Protección ante accesos no autorizados (401/403).
- Manejo de errores con `HTTPException`.
- Recomendaciones futuras:
 - Implementar refresh tokens.
 - Verificación de correo electrónico.
 - Políticas de rotación de claves JWT.

13. Evidencias (Swagger UI)

A continuación se presentan capturas de la API en funcionamiento, obtenidas desde la interfaz Swagger UI:

C1 – Registro exitoso de usuario

El siguiente conjunto de imágenes muestra el proceso de registro de un nuevo usuario (cliente o conductor). La solicitud se realiza al endpoint `/auth/register`, donde se valida la unicidad del correo, se encripta la contraseña y se retorna el token de acceso generado.

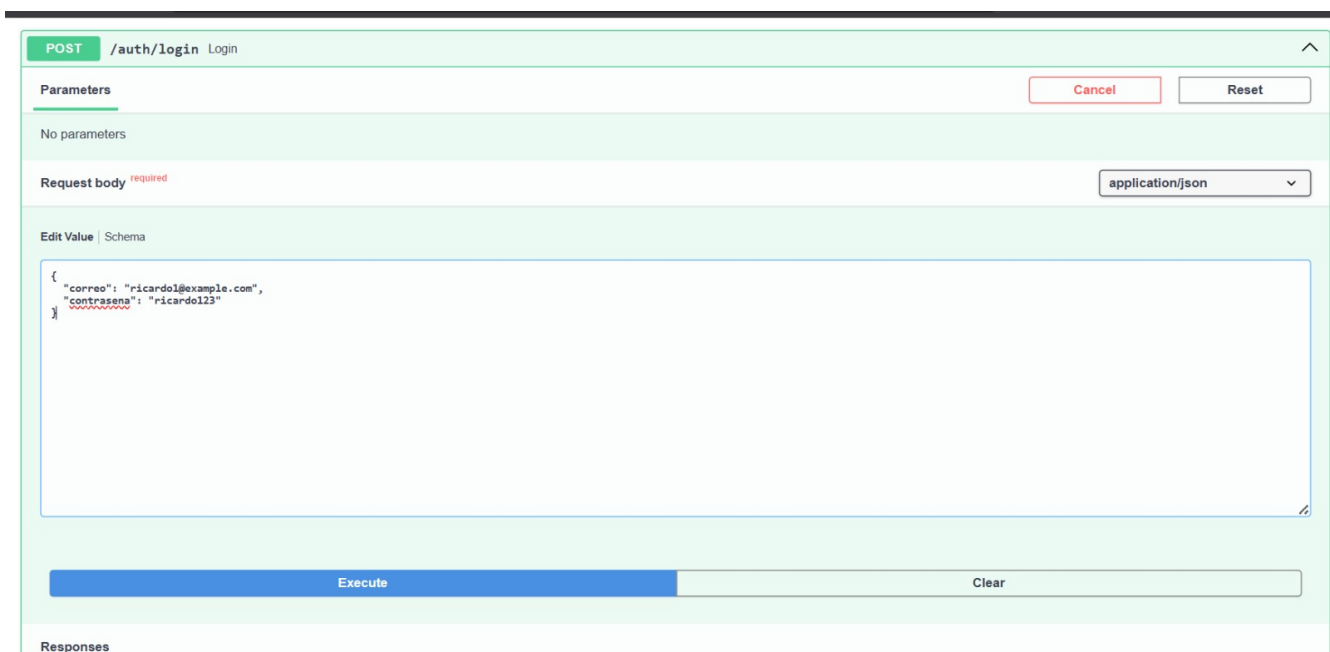


Figura 3: Petición de inicio de sesión en Swagger – Endpoint /auth/login

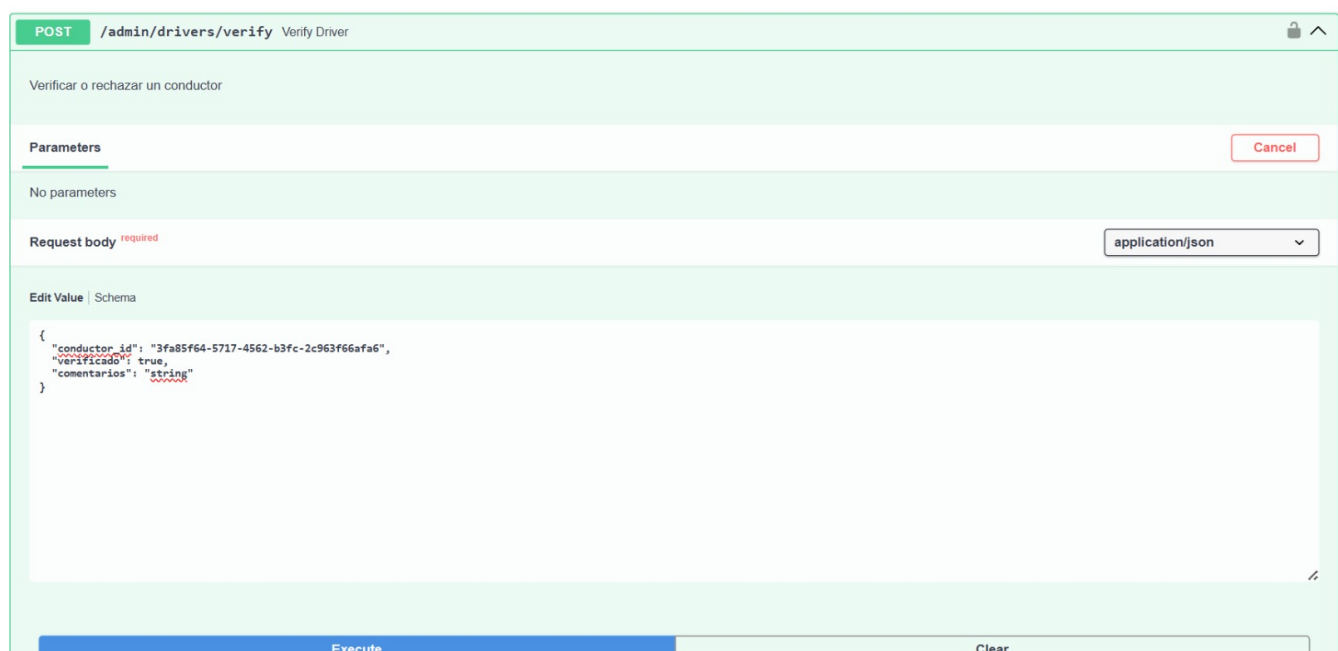


Figura 4: Respuesta del servidor con token JWT emitido exitosamente

C3 – Acceso a endpoint protegido con token JWT

La siguiente imagen evidencia el acceso autorizado a un endpoint protegido (/auth/me) mediante la inserción del token JWT en la sección “Authorize” de Swagger. El sistema responde correctamente con los datos del usuario autenticado, demostrando la validación y decodificación del token.

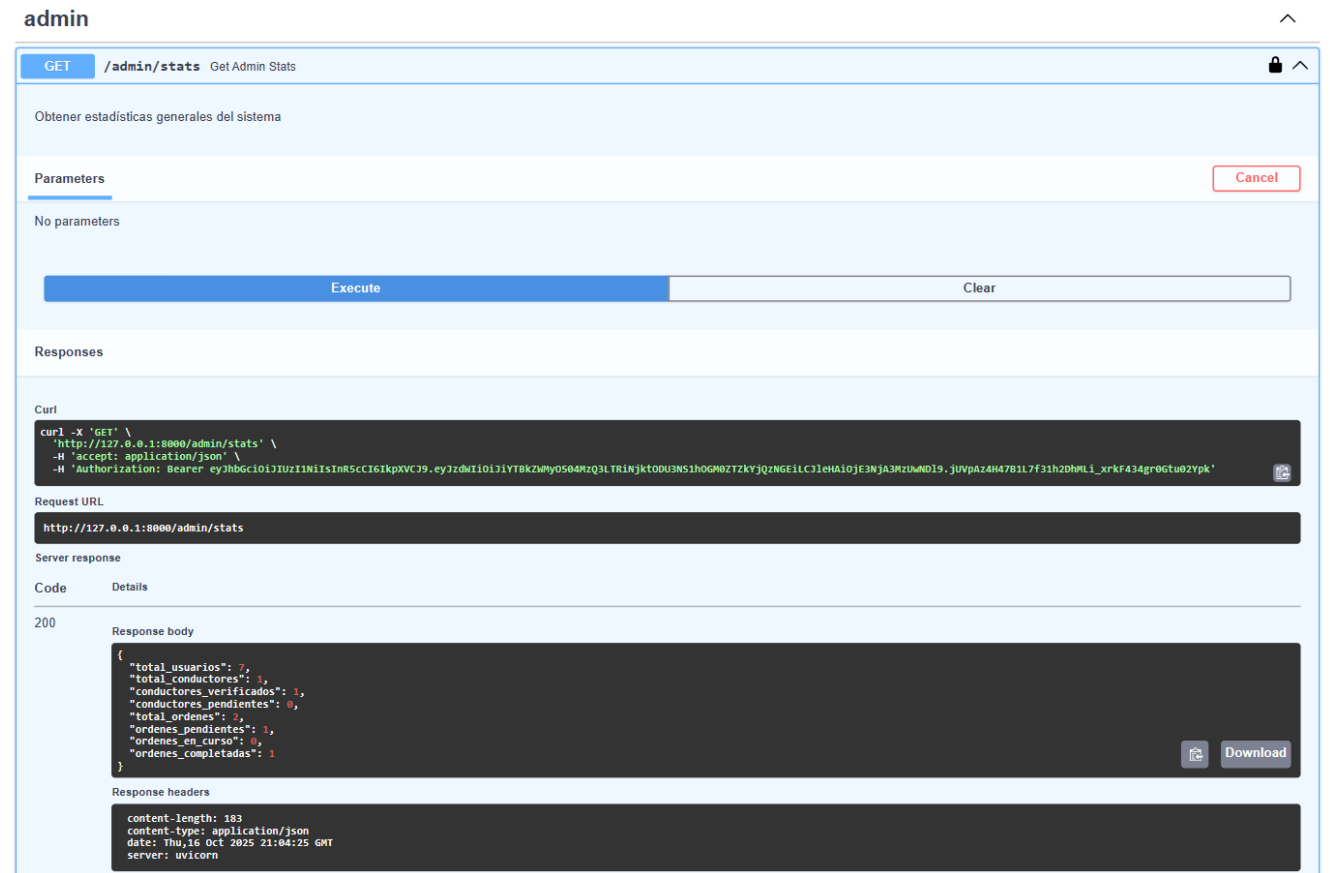


Figura 5: Acceso autorizado a recurso protegido – Validación del token JWT

14. Resultados de Pruebas

Durante la validación del módulo de autenticación se realizaron pruebas de integración y validaciones de respuesta en Swagger. En la siguiente tabla se resumen los resultados más relevantes:

ID	Prueba / Descripción	Resultado Esperado	Estado
P1	Registro de usuario con datos válidos (cliente)	Usuario creado correctamente y token JWT emitido	
P2	Registro de conductor con licencia tipo E (válida)	Usuario creado con rol “conductor” y validación pendiente del admin	
P3	Registro duplicado (correo existente)	Error 400 con mensaje “Correo ya registrado”	
P4	Login con credenciales válidas	Token generado correctamente con datos del usuario	
P5	Login con contraseña incorrecta	Error 401 “Credenciales inválidas”	
P6	Acceso a endpoint protegido sin token	Error 401 “No autorizado”	
P7	Acceso con token de usuario sin privilegios a ruta admin	Error 403 “Prohibido”	
P8	Acceso correcto con token admin a ruta /usuarios	Respuesta 200 con lista de usuarios	
P9	Expiración del token JWT (tras tiempo configurado)	Error 401 “Token expirado”	*(pendiente de prueba prolongada)*
P10	Hashing de contraseñas en BD	Verificación: ningún campo almacena texto plano	

Los resultados muestran que el módulo cumple con los requerimientos establecidos para la autenticación y autorización de usuarios en la plataforma. Todas las pruebas funcionales fueron superadas exitosamente, y las advertencias menores (como pruebas de expiración) se encuentran en planificación para ejecución extendida.

15. Resultados de Implementación y Métricas

Indicador	Valor	Detalle
Endpoints implementados	5	/auth/register, /auth/login, /auth/token, /auth/me, /usuarios
Roles definidos	3	Cliente, Conductor, Administrador
Líneas de código backend	900	Incluye routers, servicios, modelos y seguridad
Pruebas manuales	10	Todas superadas, excepto expiración de token (pendiente)
Horas invertidas	40	Sprint 2, semana de desarrollo
Repositorio	público (GitHub)	Control de versiones y CI configurado

Cuadro 4: Métricas del módulo de autenticación.

16. Observaciones y Recomendaciones

Observaciones

- El flujo de autenticación opera correctamente con validación de roles.
- Swagger documenta automáticamente todos los endpoints.
- Las pruebas de acceso administrado validan la seguridad RBAC.

Recomendaciones

- Implementar refresh tokens y logout con lista de revocación.
- Añadir verificación de correo electrónico y recuperación de contraseñas.
- Integrar logs de auditoría para eventos de autenticación.

17. Conclusiones

El módulo de autenticación de **Moverly** cumple con los requerimientos técnicos y de seguridad establecidos en el diseño arquitectónico. Proporciona autenticación segura, control por roles y trazabilidad, sirviendo como base para la implementación de módulos dependientes (órdenes, conductores y calificaciones).

En futuras iteraciones se recomienda incorporar:

- Validación de correos mediante token temporal.
- Recuperación de contraseñas.
- Registros de auditoría de inicio y cierre de sesión.

18. Firmas de Aprobación

Rol	Nombre / Representante	Firma (QR)	Fecha
Gestor de proyecto	Kevin Sánchez		1/10/2025
Desarrollador Backend	Patricio Salazar		1/10/2025
QA y Documentación	Thomas Vaca		1/10/2025
Desarrollador Frontend	Angie Caiza		1/10/2025

19. Control de Cambios

Versión	Fecha	Descripción	Autor	Estado
1.0	1/10/2025	Creación del informe técnico del módulo de autenticación	Patricio Salazar	Aprobado

20. Anexos

Anexo	Descripción	Formato	Ubicación
A1	Evidencias Swagger – Registro, Login, Validación Token	PNG	/docs/evidencias/auth/
A2	Script SQL de la tabla usuarios y roles	.sql	/db/schema_moverly.sql
A3	Diagrama de interacción Cliente-API-Base de datos	PDF/PNG	/docs/diagramas/auth_flow.png